

Neural Network from Scratch with NumPy

Hariprashad Ravikumar

1 TinyML and Efficient AI Computing

1.1 Lecture 2 - Basics of Neural Networks

Deep Neural Network

3-Layer Neural Network With 2 Hidden Layers

Synapses / Weights / Parameters

Neurons / Features / Activations

Synapse

Input Axon: x_0, x_1, x_2

Dendrite: w_0, w_1, w_2

Cell Body: $\sum_i w_i x_i + b$

Activation Function: f

Output Axon: $y_j = f\left(\sum_i w_i x_i + b\right)$

The dimensionality of these *hidden* layers determines the **width** of the model.

MIT 6.5940: TinyML and Efficient Deep Learning Computing <https://efficientml.ai> 11

Fully-Connected Layer (Linear Layer)

The output neuron is connected to all input neurons.

- **Shape of Tensors:**
 - Input Features $\mathbf{X} : (1, c_i)$
 - Output Features $\mathbf{Y} : (1, c_o)$
 - Weights $\mathbf{W} : (c_o, c_i)$
 - Bias $\mathbf{b} : (c_o,)$

$$y_i = \sum_j w_{ij} x_j + b_i$$

Notations	
c_i	Input Channels
c_o	Output Channels

$$\begin{matrix} c_i \\ \boxed{} \\ \mathbf{X} \end{matrix} \times \begin{matrix} c_o \\ \boxed{} \\ \mathbf{W}^T \end{matrix} = \begin{matrix} c_o \\ \boxed{} \\ \mathbf{Y} \end{matrix}$$

MIT 6.5940: TinyML and Efficient Deep Learning Computing <https://efficientml.ai> 13

2 Hari's personal note on Neural Networks

A Simple Neural Network

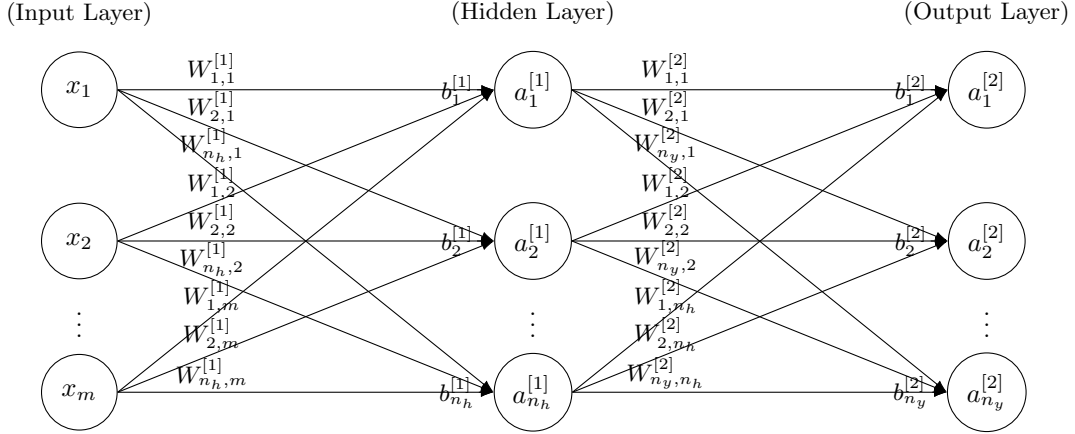


Figure 1: Simple Two Layer Neural Network (NN)

- m = number of training examples.
- $X \in \mathbb{R}^{n_x \times m}$: input data, each column $x^{(i)}$ is one example.
- $Y \in \mathbb{R}^{n_y \times m}$: output data
- For example, in MNIST, each image is $28 \times 28 = 784$ pixels; we want have output of 10 row vector $(0, 1, 2, \dots, 9)$. In this case, $n_x = 784$, $n_y = 10$. For simplicity let's choose the number of nodes in the hidden layer as $n_h = 10$

$$A^{[0]} = X \in \mathbb{R}^{784 \times m} \quad (1)$$

$$W^{[1]} \in \mathbb{R}^{10 \times 784} \quad (2)$$

$$b^{[1]} \in \mathbb{R}^{10 \times m} \quad (3)$$

- **Layer 1 (hidden):**

$$Z^{[1]} = W^{[1]} X + b^{[1]}, \quad A^{[1]} = g(Z^{[1]}),$$

where $g(\cdot)$ is an element-wise activation (e.g., ReLU or tanh).

- **Layer 2 (output):**

$$Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]}, \quad A^{[2]} = \text{softmax}(Z^{[2]}),$$

so that each column $A^{[2]}_{:,i}$ is a probability vector over n_y classes for example i .

We train with the average cross-entropy loss:

$$J = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^{n_y} Y_{k,i} \log(A^{[2]}_{k,i}). \quad (4)$$

Our goal is to compute the gradients

$$\frac{\partial J}{\partial W^{[2]}}, \quad \frac{\partial J}{\partial b^{[2]}}, \quad \frac{\partial J}{\partial W^{[1]}}, \quad \frac{\partial J}{\partial b^{[1]}}$$

so that we can update parameters by (for example) gradient descent. Below is the full chain-rule derivation.

1. Forward Propagation

1.1 Input layer $\rightarrow$ Hidden layer

$$Z^{[1]} = W^{[1]} X + b^{[1]} \in \mathbb{R}^{n_h \times m}, \quad A^{[1]} = g(Z^{[1]}) \in \mathbb{R}^{n_h \times m}.$$

Here g is applied element-wise (e.g., ReLU: $g(z) = \max(0, z)$, or $\tanh(z)$, etc.).

1.2 Hidden layer $\rightarrow$ Output layer

$$Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]} \in \mathbb{R}^{n_y \times m}, \quad A^{[2]} = \text{softmax}(Z^{[2]}) \in \mathbb{R}^{n_y \times m}.$$

Concretely, for each example i and each class k ,

$$A_{k,i}^{[2]} = \frac{\exp(Z_{k,i}^{[2]})}{\sum_{j=1}^{n_y} \exp(Z_{j,i}^{[2]})}, \quad k = 1, \dots, n_y.$$

1.3 Compute loss

$$J = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^{n_y} Y_{k,i} \log(A_{k,i}^{[2]}).$$

2. Backpropagation: Output-Layer Gradients

Because we use softmax plus cross-entropy, there is a well-known simplification:

$$\frac{\partial J}{\partial Z^{[2]}} = A^{[2]} - Y \in \mathbb{R}^{n_y \times m}. \quad (5)$$

More explicitly, for each class k and example i ,

$$[dZ^{[2]}]_{k,i} = A_{k,i}^{[2]} - Y_{k,i}.$$

We denote this matrix by

$$dZ^{[2]} \equiv A^{[2]} - Y.$$

2.1 Gradient w.r.t. $W^{[2]}$

From

$$Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]},$$

treating $A^{[1]}$ as constant,

$$\frac{\partial J}{\partial W^{[2]}} = \frac{\partial J}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W^{[2]}}.$$

In matrix form, since $Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]}$,

$$\frac{\partial J}{\partial W^{[2]}} = \frac{1}{m} (dZ^{[2]}) (A^{[1]})^T \in \mathbb{R}^{n_y \times n_h}. \quad (6)$$

(The factor $1/m$ comes from averaging over m examples.)

2.2 Gradient w.r.t. $b^{[2]}$

Because $b^{[2]}$ is broadcast-added to each column of $W^{[2]} A^{[1]}$,

$$\frac{\partial J}{\partial b^{[2]}} = \frac{1}{m} \sum_{i=1}^m [dZ^{[2]}]_{:,i} = \frac{1}{m} dZ^{[2]} \mathbf{1}_m,$$

where $\mathbf{1}_m \in \mathbb{R}^m$ is a column-vector of all ones. In practice:

$$db^{[2]} = \frac{1}{m} \sum_{i=1}^m dZ_{:,i}^{[2]} \in \mathbb{R}^{n_y \times 1}. \quad (7)$$

2.3 Gradient w.r.t. $A^{[1]}$ (propagate to previous layer)

Since $Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]}$,

$$dA^{[1]} = \frac{\partial J}{\partial A^{[1]}} = (W^{[2]})^T dZ^{[2]} \in \mathbb{R}^{n_h \times m}. \quad (8)$$

We often call this intermediate term $dA^{[1]} = (W^{[2]})^T dZ^{[2]}$.

3. Backpropagation: Hidden-Layer Gradients

At layer1, we have

$$Z^{[1]} = W^{[1]} X + b^{[1]}, \quad A^{[1]} = g(Z^{[1]}),$$

where g is the activation (e.g., ReLU or tanh). We already computed

$$dA^{[1]} = (W^{[2]})^T dZ^{[2]} \in \mathbb{R}^{n_h \times m}.$$

3.1 Compute $dZ^{[1]}$

By the chain rule,

$$dZ^{[1]} = \frac{\partial J}{\partial Z^{[1]}} = dA^{[1]} \circ g'(Z^{[1]}), \quad (9)$$

where “ $\circ$ ” denotes element-wise (Hadamard) multiplication, and

$$g'(Z^{[1]}) = \text{matrix of derivatives of } g \text{ evaluated at each entry of } Z^{[1]}.$$

- If $g = \text{ReLU}$, then

$$g'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0, \end{cases} \implies [g'(Z^{[1]})]_{j,i} = \begin{cases} 1, & Z_{j,i}^{[1]} > 0, \\ 0, & Z_{j,i}^{[1]} \leq 0. \end{cases}$$

- If $g = \tanh$, then

$$g'(z) = 1 - \tanh^2(z), \implies [g'(Z^{[1]})]_{j,i} = 1 - (A_{j,i}^{[1]})^2.$$

- If $g = \sigma$ (sigmoid), then $g'(z) = \sigma(z)(1 - \sigma(z))$.

Thus

$$dZ^{[1]} = dA^{[1]} \circ g'(Z^{[1]}) \in \mathbb{R}^{n_h \times m}.$$

3.2 Gradient w.r.t. $W^{[1]}$

From

$$Z^{[1]} = W^{[1]} X + b^{[1]},$$

treating X as constant,

$$\frac{\partial J}{\partial W^{[1]}} = \frac{\partial J}{\partial Z^{[1]}} \frac{\partial Z^{[1]}}{\partial W^{[1]}} = \frac{1}{m} (dZ^{[1]}) X^T \in \mathbb{R}^{n_h \times n_x}.$$

Hence

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T. \quad (10)$$

3.3 Gradient w.r.t. $b^{[1]}$

Because $b^{[1]}$ is added to each column of $W^{[1]}X$,

$$\frac{\partial J}{\partial b^{[1]}} = \frac{1}{m} \sum_{i=1}^m [dZ^{[1]}]_{:,i} \in \mathbb{R}^{n_h \times 1}.$$

Thus

$$db^{[1]} = \frac{1}{m} \sum_{i=1}^m dZ_{:,i}^{[1]}. \quad (11)$$

4. Full Overview

Putting it all together for a two-layer network (with a single hidden layer of size n_h):

4.1 Forward Propagation

$$\begin{cases} Z^{[1]} = W^{[1]} X + b^{[1]}, & A^{[1]} = g(Z^{[1]}), \\ Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]}, & A^{[2]} = \text{softmax}(Z^{[2]}), \end{cases}$$

$$J = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^{n_y} Y_{k,i} \log(A_{k,i}^{[2]}).$$

4.2 Backward Propagation

Output layer

$$dZ^{[2]} = A^{[2]} - Y \quad \in \mathbb{R}^{n_y \times m}, \quad (12)$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} (A^{[1]})^T \quad \in \mathbb{R}^{n_y \times n_h}, \quad (13)$$

$$db^{[2]} = \frac{1}{m} \sum_{i=1}^m dZ_{:,i}^{[2]} \quad \in \mathbb{R}^{n_y \times 1}, \quad (14)$$

$$dA^{[1]} = (W^{[2]})^T dZ^{[2]} \quad \in \mathbb{R}^{n_h \times m}. \quad (15)$$

Hidden layer

$$dZ^{[1]} = dA^{[1]} \circ g'(Z^{[1]}) \quad \in \mathbb{R}^{n_h \times m}, \quad (16)$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T \quad \in \mathbb{R}^{n_h \times n_x}, \quad (17)$$

$$db^{[1]} = \frac{1}{m} \sum_{i=1}^m dZ_{:,i}^{[1]} \quad \in \mathbb{R}^{n_h \times 1}. \quad (18)$$

4.3 Parameter Updates (Gradient Descent)

After computing these gradients, update each parameter by

$$W^{[\ell]} \leftarrow W^{[\ell]} - \alpha dW^{[\ell]}, \quad b^{[\ell]} \leftarrow b^{[\ell]} - \alpha db^{[\ell]}, \quad \ell = 1, 2,$$

where α is the learning rate.

5. Generalization to More Layers

If there are L layers, the pattern is:

$$Z^{[\ell]} = W^{[\ell]} A^{[\ell-1]} + b^{[\ell]}, \quad A^{[\ell]} = g^{[\ell]}(Z^{[\ell]}), \quad \ell = 1, \dots, L,$$

with $A^{[0]} = X$. The final layer L uses softmax (for multiclass) or sigmoid (for binary).

During backpropagation:

Compute $dZ^{[L]}$ from the loss derivative at the output (e.g., $A^{[L]} - Y$).

For $\ell = L, L-1, \dots, 2$:

$$dW^{[\ell]} = \frac{1}{m} dZ^{[\ell]} (A^{[\ell-1]})^T,$$

$$db^{[\ell]} = \frac{1}{m} \sum_{i=1}^m dZ_{:,i}^{[\ell]},$$

$$dA^{[\ell-1]} = (W^{[\ell]})^T dZ^{[\ell]},$$

$$dZ^{[\ell-1]} = dA^{[\ell-1]} \circ g'^{[\ell-1]}(Z^{[\ell-1]}).$$

Then update each $W^{[\ell]}, b^{[\ell]}$ accordingly.

References

Michael A. Nielsen, *Neural Networks and Deep Learning*, Determination Press, 2015.